Analogy to ICSM

- A typical total commitment game is ***roulette***: you put all your chips (money) on a number and wait to see whether you win or lose.

- In a software project, you would bet your development budget on the on-time delivery of your product, and then wait and see whether you will make it.

Analogy to ICSM

- Typical incremental commitment games are **poker** or **blackjack**. You put some chips in, see your cards and some of the others' cards, and then you decide whether and how much more money to commit.
- If your cards are too bad, you bail out, avoiding losing more money in a situation where it is highly unlikely for you to win.
- The ICSM enables a similar strategy for software projects: You spend some of your development budget and check the results (for example, whether your application interfaces correctly with another application) in order to find out whether and how to progress.

Prerequisites

- A necessary precondition to applying the ICSM is the understanding of all involved partners and their willingness to stop a project if the risk analysis suggests that.
- In other words, the ICSM cannot be applied in projects that must deliver a result, no matter how good (or bad) it is.
- Applying the ICSM also requires the capability to identify and assess risks.

Advantages

- The ICSM prevents premature total commitments, which can lead to a situation where there is no way back.
- The early identification of project risks and either their mitigation or the adjustment of the project scope facilitates the congruence of expected and actual project results.
- Additionally, the ICSM opens an exit strategy to stop a project before burning away lots of money, without the need for anyone to lose their face—which can be highly beneficial in projects where politics play an important role.

Challenges

- Honestly assessing risks requires great openness within an organization.
- This may be difficult for certain employee evaluation models. The systems addressed by the ICSM (large, high-risk) are sometimes political projects that normally cannot be stopped, no matter how bad the situation is—in these cases, a cultural change both on the buyer and on the supplier side is required.

- *The ICSM was developed for very large, very risky projects where one does not know at the beginning whether and to what extent the goal can be reached.*

Example Lifecycle Models

- What we have discussed are Lifecycle models in conceptual form.
- Those generic conceptual models can be adopted to practice
- In practice there are example lifecycle models.



Unified Process

- Published by Jacobson, Booch, and Rumbaugh in 1999
- Unified Process is a generic process framework for software development.
- The Unified Process distinguishes project phases like the waterfall model does and supports iterative development within the phases like the iterative enhancement model does.
- It also specifically considers risks during the iterations, borrowing from the spiral model.
- In addition, it contains traces of the prototyping model, since it recommends building prototypes within iterations under special circumstances.

UP

- The Unified Process is component based, i.e., the system to be built is compiled from software components that are interconnected via well-defined interfaces.
- The Unified Modelling Language (UML) is used to model all central artefacts describing the system—e.g., requirements and design.
- Unified Process is use-case driven. Within the Unified Process, use cases replace the traditional functional specification of the system.
- The Unified Process focuses on the user who will work with the system, as opposed to the system itself.
- Use cases are central to system development, also driving design, implementation, and test.

UP

- The Unified Process is also architecture-centric, i.e., it focuses strongly on the system architecture, embodying the most significant static and dynamic aspects of the system.
- The architecture is influenced by the use cases, because it must satisfy all the stated user needs.
- It is also influenced by external factors such as the platform.
- The more detailed and mature the use cases become, the more details are added to the architecture, which in turn facilitates the maturation of additional use cases.

UP

- The Unified Process is iterative and incremental.
- The Unified Process divides the work into mini projects forming iterations and resulting in increments that are added to the system.
- Each iteration contains the core workflows requirements, analysis, design, implementation, and test.

- Not all iterations add functionality; for example, the system architecture may be modified in one increment without adding any new features.

UP Phases

- A project typically goes through the phases Inception, Elaboration, Construction, and Transition. The phases are separated by milestones, defined by the availability of a set of artefacts (i.e., certain models or documents are in a prescribed state).



- The inception phase creates a vision of and a business case for the final system.
- During the elaboration phase, most of the system's use cases are specified and its architecture is defined.
- The construction phase builds the system, evolving the architecture to its final state.
- Finally, the transition phase covers the system's beta testing, correcting the defects found by the beta testers.

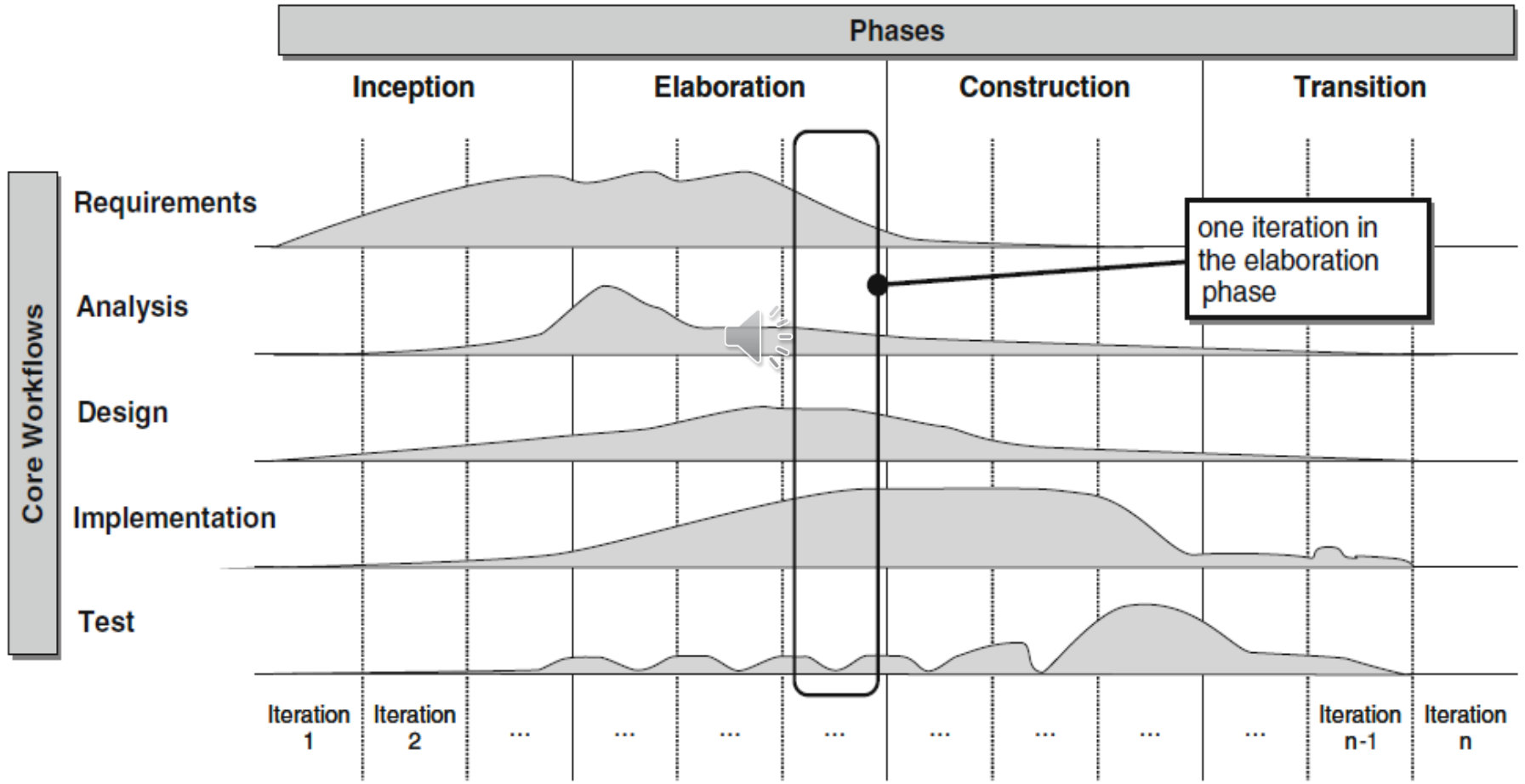


Fig. 2.9 The core workflows and phases of the Unified Process

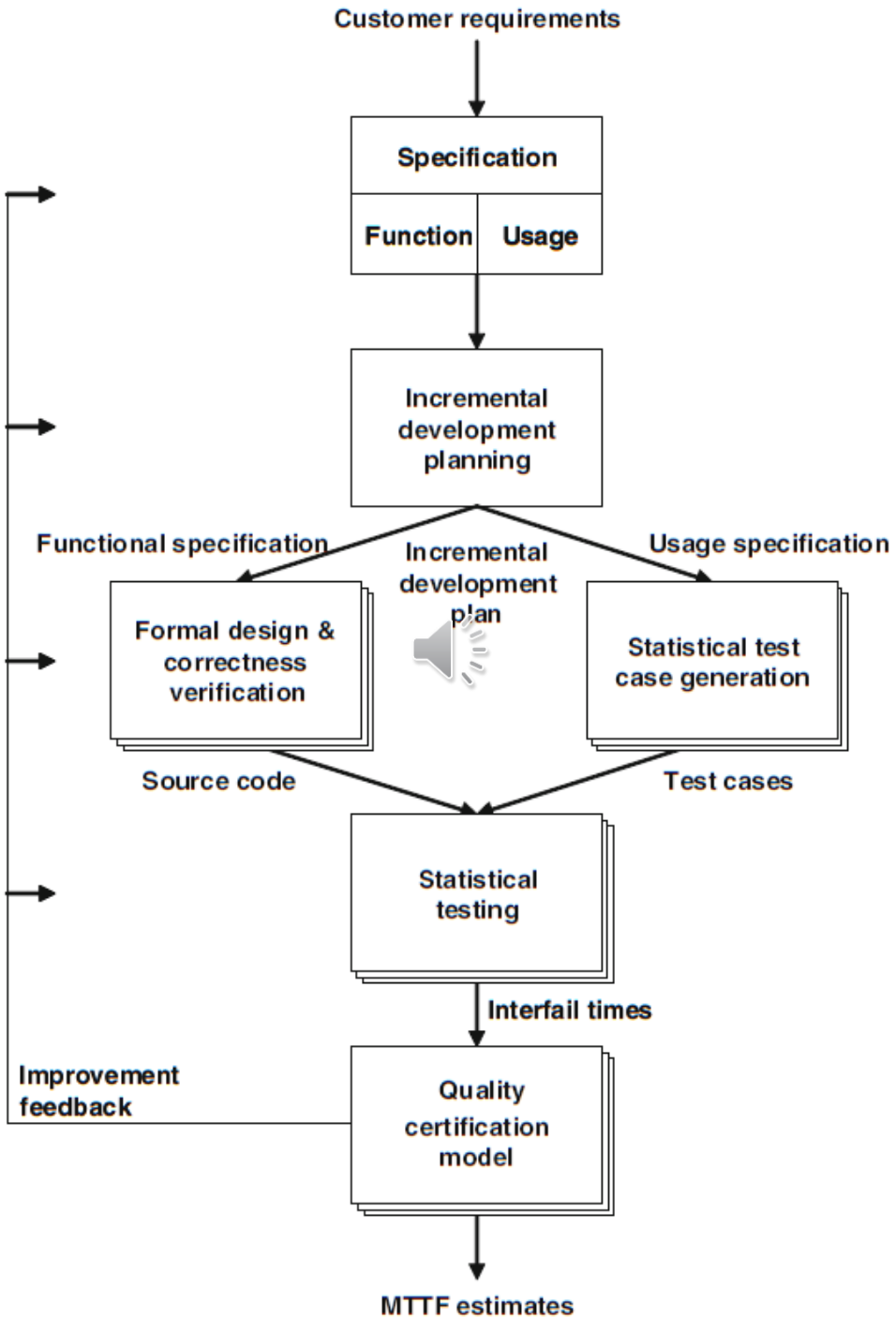
UP Application

- The Unified Process is a popular lifecycle model; however, its application requires some process maturity, e.g., to clearly distinguish the project phases and the development activities.
- The iterative nature within the phases may be confusing, because the phases make the Unified Process look like a waterfall process.
- Another challenge is the fact that the Unified Process “stops” when the system is in beta testing, so actually bringing it into service and operation must be done using other means— a detail that may easily be forgotten.
- The Unified Process is a comprehensive development process that must be adapted to the specific organization before it can be applied.

Example 2 – Cleanroom Development

- Published by Mills, Dyer, and Linger in 1987.
- The Cleanroom Development Process distinguishes itself from other approaches by focusing on the formal verification of correctness, as opposed to trying to eliminate as many defects as possible by testing a system.
- The Cleanroom Development 
- Process is based on developing and certifying a sequence of software increments that together form the final system.
- Each increment is developed and certified by a small, independent team.
- The development teams assure correctness via formal specification, design, and verification.

- The team correctness verification takes the place of testing and debugging—the development team does not execute the code that it writes!
- The Cleanroom Development Process is therefore an iterative process, similar to the iterative enhancement model.


Customer requirements
Specification
Function
Usage
Incremental development planning
Functional specification
Incremental development plan
Usage specification
Formal design & correctness verification
Statistical test case generation
Source code
Test cases
Statistical testing
Interfail times
Improvement feedback
Quality certification model
MTTF estimates

CR development

- The functional specification defines the required external system behaviour in all circumstances of use in a formal way;
- The usage specification defines usage scenarios and their probabilities for all possible system usages, both correct and incorrect ones.
- While the functional specification is the basis for the incremental system development, the usage specification is the basis for generating test cases.
- On the basis of both specifications, the development and certification teams together define an initial plan for incrementally developing the system.

- Each completed increment is integrated with the system (i.e., all prior increments) and delivered to the statistical test team.
- Statistical measures are taken to check the correctness against the functional specification (e.g. Mean time to failure MTTF)
- All errors found during testing are returned to the development team for correction.

CR Development

- The Cleanroom Development Process has proven to be able to produce high quality (in terms of number of defects) software.
- However, it is not widely adopted.
- In order to correctly develop a software system with Cleanroom, all involved developers must be highly skilled, especially with respect to formal specification and verification.
- Scalability – LoC challenge
- The CRD Process is capable of delivering high-quality software; however, it requires extremely skilled process performers, and its scalability for large systems is unknown.